

Lecture 3: Conditioning and Guidance

Cole Becker

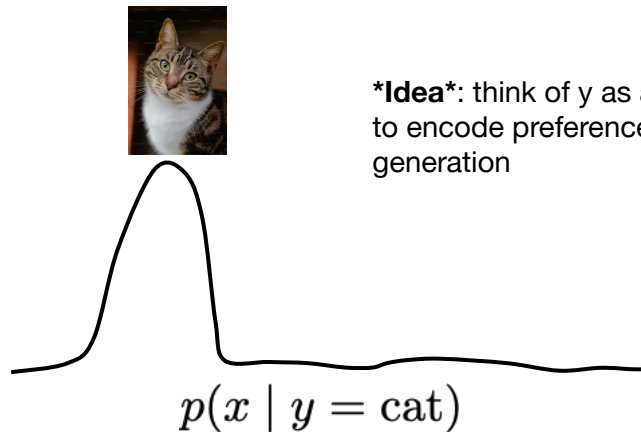
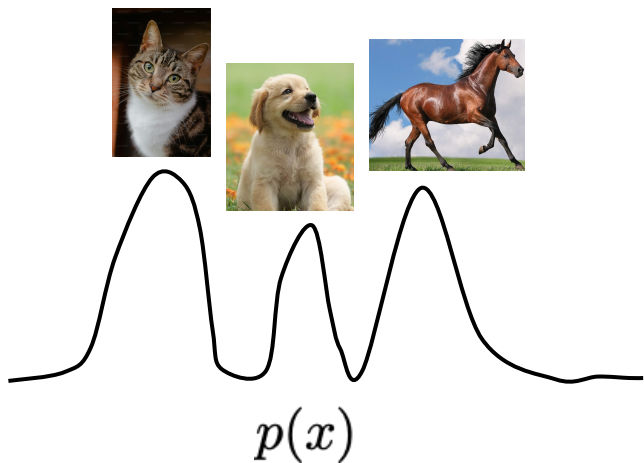
January 9th, 2026



Conditional Diffusion (overview)

Previously: sample from $p(x)$

Today: Sample from $p(x \mid y)$ for some condition y



***Idea*:** think of y as a way to encode preferences in generation

Outline

1. Recap: denoiser to score function
2. Training (unconditional) score function
3. Unconditional to conditional generation and training
4. Classifier guidance and Classifier free guidance
5. Introduction to Inverse Problems



Outline

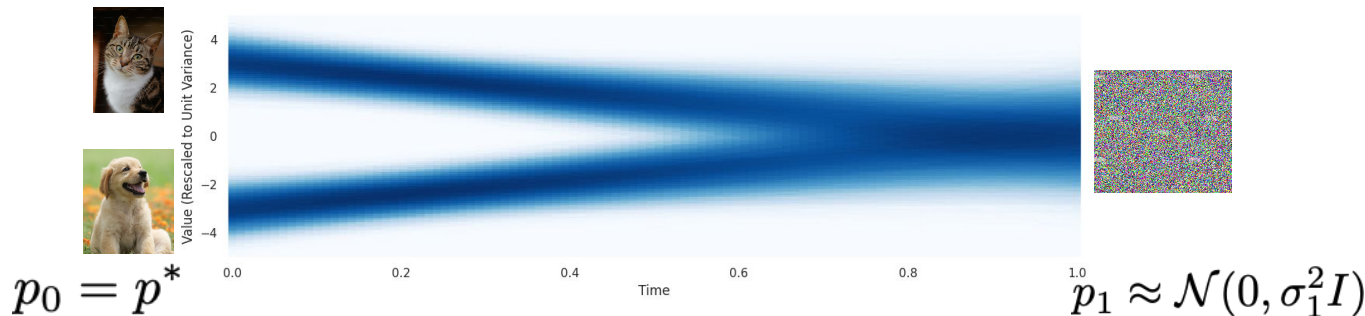
1. **Recap: denoiser to score function**
2. Training (unconditional) score function
3. Unconditional to conditional generation and training
4. Classifier guidance and Classifier free guidance
5. Introduction to Inverse Problems



Recap: building probability path

Noising process: $x_t \stackrel{\text{dist}}{=} x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

→ Induces time-varying marginal $p_t(x_t)$ with $p_0 = p^* \quad p_1 \approx \mathcal{N}(0, \sigma_1^2 I)$



Recap: Deterministic vs Stochastic Sampling

Noising process: $x_t \stackrel{\text{dist}}{=} x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

→ Induces time-varying marginal $p_t(x_t)$ with $p_0 = p^*, p_1 \approx \mathcal{N}(0, \sigma_1 I)$

Continuous ODE-Flow

$$\dot{x}_t = v(x_t, t)$$

$$v(x_t, t) = \dot{\sigma}(t) \mathbb{E}[\epsilon \mid x_t]$$

Continuous (Backwards) SDE

$$dx = f(x, t) \overleftarrow{dt} + \sqrt{2\rho(t)} \overleftarrow{dB}_t$$

$$f(x, t) = \left(\dot{\sigma}(t) - \frac{\rho(t)}{\sigma(t)} \right) \mathbb{E}[\epsilon \mid x_t]$$

Key point 1



Both methods traverse $p_t(x)$ when using $x_1 \sim \mathcal{N}(0, \sigma_1^2 I)$ approximation

Key point 2



Having access to $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$, for $t \in [0, 1]$ is sufficient to sample



From denoiser to score function

Noising process: $x_t \stackrel{\text{dist}}{=} x_0 + \sigma_t \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ induces $p_t(x_t)$, $t \in [0, 1]$

Up until now we have been considering $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$, to sample

However, recall we showed $\mathbb{E}[\epsilon \mid x_t] = -\sigma_t \nabla \log p_t(x_t)$

 To sample, it is equivalent to have access to $s^*(x_t, t) := \nabla \log p_t(x_t)$

We will work with $\nabla \log p_t(x_t)$ for the remainder of the lecture

Outline

1. Recap: denoiser to score function
2. **Training (unconditional) score function**
3. Unconditional to conditional sampling and training
4. Classifier guidance and Classifier free guidance
5. Introduction to Inverse Problems



Loss function to learn $\nabla \log p_t(x_t)$?

Noising process: $x_t \stackrel{\text{dist}}{=} x_0 + \sigma_t \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ induces $p_t(x_t)$, $t \in [0, 1]$

Input: $D = \{x_0^{(i)}\}_{i=1}^N$

$$\mathcal{L}_{\text{naive}}(\theta) = \mathbb{E}_{t \sim \text{Unif}[0,1], x_0 \sim p_0, x_t \sim p_{t|0}(\cdot | x_0)} \|s_\theta(x_t, t) - \nabla_x \log p_t(x_t)\|^2$$

Obvious problem: no closed form for $\nabla_x \log p_t(x_t)$

However: do know $p_{t|0}(x_t | x_0) = \mathcal{N}(x_0, \sigma_t^2 I)$

$$\implies \nabla \log p_{t|0}(x_t | x_0) = -\frac{x_t - x_0}{\sigma_t^2}$$



Score Matching Derivation (tweedies revisited)

Step 1: Can we express $\nabla_x \log p_t(x_t)$ in terms of $\nabla \log p_{t|0}(x_t | x_0)$?

$$\begin{aligned}\nabla \log p_t(x_t) &= \frac{\nabla p_t(x_t)}{p_t(x_t)} \\&= \frac{1}{p_t(x_t)} \nabla_x \int p_{t|0}(x_t | x_0) p_0(x_0) dx_0 \\&= \frac{1}{p_t(x_t)} \int \nabla_x p_{t|0}(x_t | x_0) \frac{p_{t|0}(x_t | x_0)}{p_{t|0}(x_t | x_0)} p_0(x_0) dx_0 \\&= \int \nabla_x \log p_{t|0}(x_t | x_0) \frac{p_{t|0}(x_t | x_0) p_0(x_0)}{p_t(x_t)} dx_0 \\&= \int \nabla_x \log p_{t|0}(x_t | x_0) p_{0|x_t}(x_0 | x_t) dx_0 \\&= \mathbf{E}_{x_0 \sim X_0 | X_t = x_t} [\nabla_x \log p_{t|0}(x_t | x_0)]\end{aligned}$$

Score Matching Derivation cont.

Previous slide : $\nabla \log p_t(x_t) = \mathbf{E}_{x_0 \sim X_0 | X_t = x_t} [\nabla_x \log p_{t|0}(x_t | x_0)] = \frac{\mathbf{E}[x_0 | x_t] - x_t}{\sigma_t^2}$

Recall from hw: $\operatorname{argmin}_{f(u)} \mathbb{E}_{(x,u)} [\|x - f(u)\|^2] = \mathbb{E}_{x|u}[x | u]$

$$\operatorname{argmin}_{f(u)} \mathbb{E}_{(x,u)} [\|g(x, u) - f(u)\|^2] = \mathbb{E}_{x|u}[g(x, u) | u]$$

$$\begin{aligned} \Rightarrow \operatorname{argmin}_{s_{\theta}(x_t, t)} \mathbb{E}_{t, x_t, x_0} [\|\nabla \log p_t(x_t | x_0) - s_{\theta}(x_t, t)\|^2] &= \mathbb{E}_{x_0|x_t} [\nabla \log p_{t|0}(x_t | x_0)] \\ &= \nabla \log p_t(x_t) \end{aligned}$$

Loss function to learn $\nabla \log p_t(x_t)$?

Noising process: $x_t \stackrel{\text{dist}}{=} x_0 + \sigma_t \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ induces $p_t(x_t)$, $t \in [0, 1]$

Input: $D = \{x_0^{(i)}\}_{i=1}^N$

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}[0,1], x_0 \sim p_0, x_t \sim p_{t|0}(\cdot | x_0)} \|s_\theta(x_t, t) - \nabla_x \log p_{t|0}(x_t | x_0)\|^2$$

Takeaway: The score matching loss function is a tractable way to learn unconditional score function



Outline

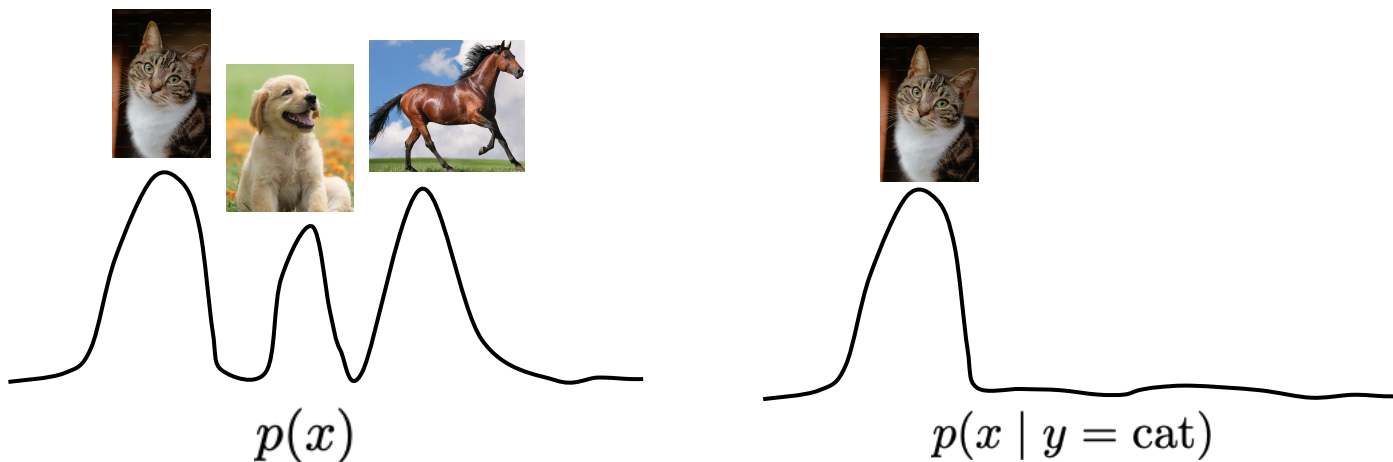
1. Recap: denoiser to score function
2. Training (unconditional) score function
3. **Unconditional to conditional sampling and training**
4. Classifier guidance and Classifier free guidance
5. Introduction to Inverse Problems



Conditional Diffusion

We know we can use $\nabla \log p_t(x_t)$ to sample ~randomly~ from p_0

Q: how can I sample conditionally? How can I incorporate preferences?



Conditional Diffusion

A: We can just do the same thing! But with labeled data (x, c)

Input: $D = \{(x_0, c)^{(i)}\}_{i=1}^N$

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{(x_0, c) \sim p_0, t \sim \text{Unif}[0, 1], x_t \sim p_{t|0}(\cdot | x_0)} \|s_t^\theta(x_t, c) - \nabla_x \log p_{t|0}(x_t | x_0)\|^2 p_0$$

Takeaway: Instead of learning $\nabla \log p_t(x_t)$
We learn $\nabla \log p_t(x_t | c)$



Unconditional and Conditional sampling are the same

Consider ODE-Flow

Unconditional Generation

$$\dot{x}_t = v(x_t, t)$$

$$v(x_t, t) = -\dot{\sigma}_t \sigma_t \nabla \log p_t(x_t)$$

Conditional Generation

$$\dot{x}_t = v_c(x_t, t)$$

$$v_c(x_t, t) = -\dot{\sigma}_t \sigma_t \nabla \log p_t(x_t | c)$$



Does this work?

Yes and no.. Works in some cases, usually when $\mathbf{x}$ is low dimensional



Let's generate some photos of dogs. Doesn't work very great!

Outline

1. Recap: denoiser to score function
2. Training (unconditional) score function
3. Unconditional to conditional sampling and training
4. **Classifier guidance and Classifier free guidance**
5. Introduction to Inverse Problems



Classifier Guidance (Dhariwal Nicol 2021)

Idea 1: learn an unconditional score function and a classifier

$$p_t(x_t | c) = \frac{p_t(x_t)p_t(c | x_t)}{p(c)}$$

$$\implies \nabla_x \log p_t(x_t | c) = \underbrace{\nabla_x \log p_t(x_t)}_{\text{score function}} + \nabla_x \log \underbrace{p_t(c | x_t)}_{\text{classifier}}$$

Idea 2: what happens if we scale up the 2nd term by a “guidance parameter” $\gamma > 1$?

$$\nabla_x \log \tilde{p}_t(x_t | c) := \underbrace{\nabla_x \log p_t(x_t)}_{\text{score function}} + \gamma \nabla_x \log \underbrace{p_t(c | x_t)}_{\text{classifier}}, \quad \gamma > 1$$

Classifier-Free Guidance (Ho Salimans, 2022)

Same idea, just train one conditional score function instead with an extra class $c = \emptyset$

$$\begin{aligned}\nabla_x \log \tilde{p}_t(x_t | c) &:= \underbrace{\nabla_x \log p_t(x_t)}_{\text{score function}} + \gamma \underbrace{\nabla_x \log p_t(c | x_t)}_{\text{classifier}}, \quad \gamma > 1 \\ &= \nabla_x \log p_t(x_t) + \gamma (\nabla_x \log p_t(x_t | c) - \nabla_x \log p_t(x_t)) \\ &= (1 - \gamma) \nabla_x \log p_t(x_t) + \gamma \nabla \log p_t(x | c) \\ &= (1 - \gamma) \nabla_x \log p_t(x_t | c = \emptyset) + \gamma \nabla \log p_t(x | c)\end{aligned}$$

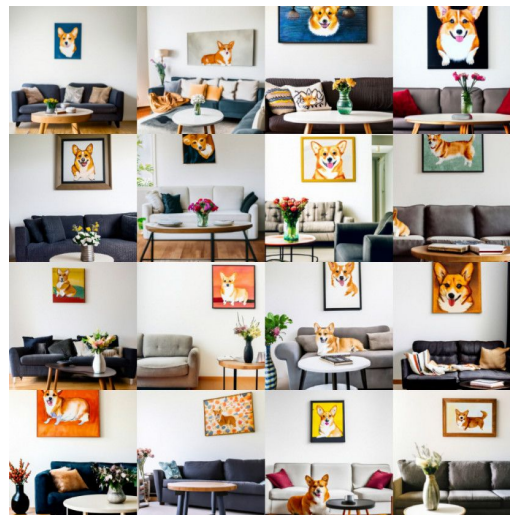
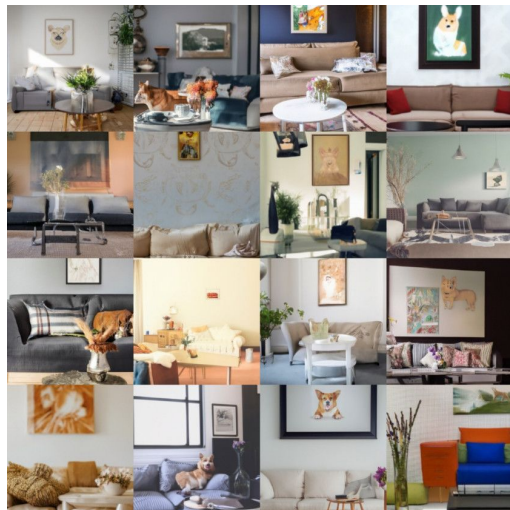
Takeaway: training is identical to training conditional diffusion model, but with a dummy class $c = \emptyset$ used during training as well.

Classifier-free Guidance → sample quality higher



OpenAI's GLIDE model, without guidance (left) and with guidance (right)

Classifier-free Guidance → sample quality higher

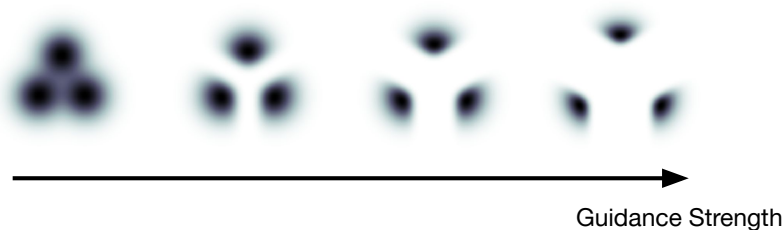
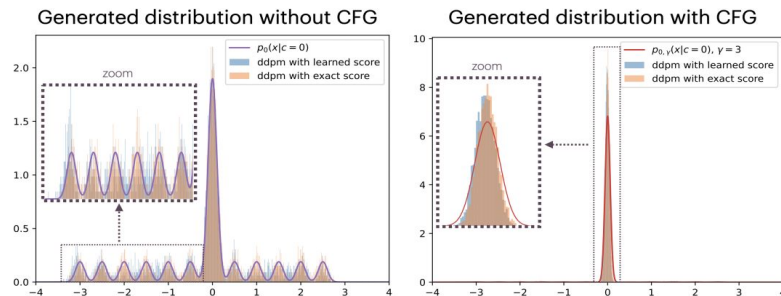


OpenAI's GLIDE model, without guidance (left) and with guidance (right)

What is Classifier-free Guidance “doing”?

Theory is still developing but empirically:

1. Increasing guidance γ leads to sharper “low-temperature” sampling (i.e. higher density regions of distribution contain higher perceived quality samples)
2. Learned score functions tend to have more significant error in low density regions, and so this error is less apparent with guidance



CFG is a predictor corrector, Bradley and Nakkiran, 2024, and Guiding a Diffusion Model with a Bad Version of Itself, Karras et al. 2024

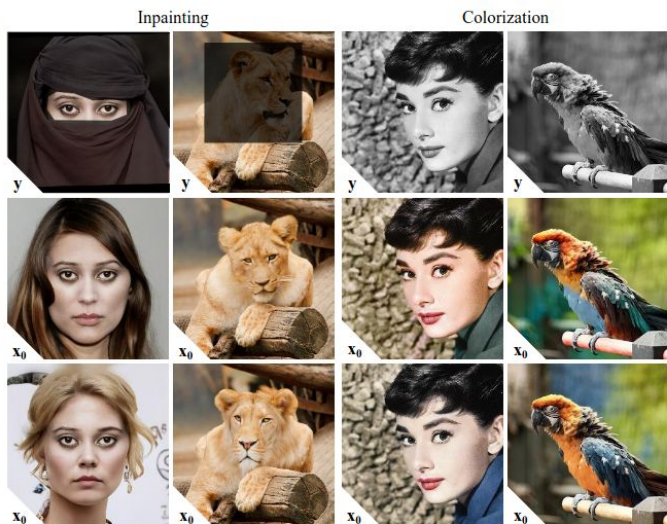
Outline

1. Recap: denoiser to score function
2. Training (unconditional) score function
3. Unconditional to conditional sampling and training
4. Classifier guidance and Classifier free guidance
5. **Introduction to Inverse Problems**



Inverse Problems: Setup

Goal: recover samples $x_0 \sim p_0$ given access to measurements y following corruption model $y = \mathcal{A}(x_0) + \sigma_y z$, $z \sim \mathcal{N}(0, I)$



Inverse Problems: brief overview

- diffusion models often seek to accomplish this task by approximating the conditional score $\nabla_x \log p_t(x | y)$ in order to sample from $p(x | y)$

Naive Solution:

- Gather (x,y) pairs and train a conditional diffusion model (and use CFG)
- This works, but not very flexible and need to retrain if you wish to solve a different inverse problem.

Common Solution:

- Recall bayes rule for score functions $\nabla_x \log p_t(x_t | y) = \underbrace{\nabla_x \log p_t(x_t)}_{\text{unconditional score}} + \underbrace{\nabla_x \log p_t(y | x_t)}_{\text{measurement-matching}}$
- Train unconditional score function and approximate measurement term for different corruption models
- More flexible, but not always easy to approximate measurement term



Feedback/Attendance Survey



<https://forms.gle/LsQJthBmxW9CsVNc7>



Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models